CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 4 Dynamic Programming

ملخص الميد · CLRS Chapter 15

محارب

محتوى الملف

01 الفكرة

02 متى نستخدم DP؟ (الخاصيتين)

03 Matrix-chain multiplication

04 مثال: أربع مصفوفات (السلايدات)

05 مثال: ست مصفوفات (السلايدات)

06 Longest common subsequence (LCS)

01 الفكرة

الـ DP يحل المسألة من مسائل أصغر، ويخزن نتيجة كل مسألة صغيرة في جدول عشان ما يعيد حسابها.
يشتغل من الأصغر للأكبر (bottom-up).

Dynamic programming is an optimization method based on the **principle of optimality** (Bellman, 1950s):
"Every optimal policy consists only of optimal sub-policies."

مثال Fibonacci

```
F(n) = F(n−1) + F(n−2)
```

```
Recursive:  each call makes 2 calls  →  O(2ⁿ)
```

```
            (a lot of repeated work)
```

```
DP:         fib[1] = fib[2] = 1
```

```
            for i = 3 to n:  fib[i] = fib[i−1] + fib[i−2]
```

```
            →  O(n)
```

02 متى نستخدم DP؟ (الخاصيتين)

الخاصية	معناها
Optimal sub-structure	الحل الأمثل للمسألة فيه حلول مثلى لمسائلها الأصغر.
Overlapping sub-problems	نفس المسألة الصغيرة تتكرر أكثر من مرة.

الفرق عن Divide and conquer: هناك المسائل الصغيرة مستقلة وما تتكرر. في الـ DP تتكرر، فنخزنها.

الخطوات الأربع

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution, typically bottom-up.
4. Construct an optimal solution from the computed information.

التطبيقات: **Matrix-chain multiplication** و **Longest common subsequence** و **Optimal binary search trees**.

جاء في الميد

An algorithm which uses the past results and uses them to find new results is → **Dynamic programming**

When DP is applied, it takes far less time than methods that don't take advantage of overlapping sub-problems. → **True**

Greedy algorithm can solve all the dynamic programming problems. → **False**

Matrix-chain multiplication 03

عندنا مصفوفات نبي نضربها. الترتيب ما يغير الناتج، بس يغير عدد عمليات الضرب. المطلوب: أقل عدد عمليات، وطريقة التقويس (parenthesization).

```
A (p×q) · B (q×r)  =  C (p×r)        cost = p · q · r
```

مثال السلايد: ثلاث مصفوفات

```
A = 10×100,   B = 100×5,   C = 5×50
```

```
((AB)C):  AB = 10·100·5 = 5,000
```

```
          (10×5)·C = 10·5·50 = 2,500
```

```
          total = 7,500
```

```
(A(BC)):  BC = 100·5·50 = 25,000
```

```
          A·(100×50) = 10·100·50 = 50,000
```

```
          total = 75,000
```

نفس النتيجة بعشر العمليات. عشان كذا نحتاج DP يلقى أحسن ترتيب.

المعادلة

```
m[i, i] = 0
```

```
m[i, j] = min over i ≤ k < j of
```

```
          m[i, k] + m[k+1, j] + p(i−1)·p(k)·p(j)
```

```
s[i, j] = the k that gave the minimum
```

`p` = قائمة الأبعاد: المصفوفة `Ai` أبعادها `p(i−1) × p(i)` . نعبي الجدول من السلاسل القصيرة (طول 2) لين الطويلة.

04 مثال: أربع مصفوفات (السلايدات)

```
A0 = 10×5,  A1 = 5×8,  A2 = 8×4,  A3 = 4×10
```

```
Length 2:
A0A1 = 10·5·8 = 400
A1A2 = 5·8·4 = 160
A2A3 = 8·4·10 = 320
Length 3:
A0(A1A2) = 160 + 10·5·4 = 360   ✓
(A0A1)A2 = 400 + 10·8·4 = 720
A1(A2A3) = 320 + 5·8·10 = 720
(A1A2)A3 = 160 + 5·4·10 = 360   ✓
Length 4:
A0(A1A2A3)    = 360 + 10·5·10 = 860
(A0A1)(A2A3)  = 400 + 320 + 10·8·10 = 1,520
(A0A1A2)A3    = 360 + 10·4·10 = 760     ✓
```

	A0	A1	A2	A3
A0	0	400	360	**760**
A1	—	0	160	360
A2	—	—	0	320
A3	—	—	—	0

```
Minimum = 760        Order: ((A0(A1A2))A3)
```

السلايد كاتب `(A0A1)(A2A3) = 400 + 360 + 800 = 1560`، بس `A2A3 = 8·4·10 = 320`، فالصحيح `1520`.
الجواب الأقل (760) ما يتغير.

لما نضيف `A4 = 10×6` يصير الجواب 820 بالترتيب `A0((A1A2)(A3A4))`:

	A0	A1	A2	A3	A4
A0	0	400	360	760	**820**
A1	–	0	160	360	520
A2	–	–	0	320	432
A3	–	–	–	0	240
A4	–	–	–	–	0

05 مثال: ست مصفوفات (السلايدات)

```
A1 = 10×20, A2 = 20×5, A3 = 5×15, A4 = 15×50, A5 = 50×10, A6 = 10×15
```

```
p = 10, 20, 5, 15, 50, 10, 15
```

```
m[1,3] = min{ 0 + 1500 + 10·20·15 , 1000 + 0 + 10·5·15 }
       = min{ 4500, 1750 } = 1750
m[2,4] = min{ 0 + 3750 + 20·5·50 , 1500 + 0 + 20·15·50 }
       = min{ 8750, 16500 } = 8750
m[3,5] = min{ 0 + 7500 + 5·15·10 , 3750 + 0 + 5·50·10 }
       = min{ 8250, 6250 } = 6250
m[4,6] = min{ 0 + 7500 + 15·50·15 , 7500 + 0 + 15·10·15 }
       = min{ 18750, 9750 } = 9750
```

	A1	A2	A3	A4	A5	A6
A1	0	1000	1750	7250	7750	**8750**
A2	–	0	1500	8750	7250	8500
A3	–	–	0	3750	6250	7000
A4	–	–	–	0	7500	9750
A5	–	–	–	–	0	7500
A6	–	–	–	–	–	0

جدول `s` يقول وين نقسم. `s[1,6] = A2` يعني `(A1A2)(A3…A6)`، وبعدها `s[3,6] = A5` و `s[3,5] = A4`:

s	A1	A2	A3	A4	A5	A6
A1	–	A1	A2	A2	A2	A2
A2	–	–	A2	A2	A2	A2
A3	–	–	–	A3	A4	A5
A4	–	–	–	–	A4	A5
A5	–	–	–	–	–	A5
A6	–	–	–	–	–	–

```
Minimum = 8750        Order: ((A1A2)(((A3A4)A5)A6))
```

Longest common subsequence (LCS) 06

الـ subsequence: حروف من الكلمة بنفس ترتيبها، مو لازم ورا بعض. مثال: من `ABACDABAB` : `ABA` و `ACA` صح، و `DCA` غلط (الترتيب مقلوب).

```
c[i, 0] = c[0, j] = 0
```

```
X[i] = Y[j]:   c[i, j] = c[i-1, j-1] + 1        diagonal + 1   (↖)
```

```
X[i] ≠ Y[j]:   c[i, j] = max(c[i-1, j], c[i, j-1])
```

```
               larger of top (↑) and left (←)
```

الجدول يتعبى صف صف. الجواب في آخر خانة. وعشان نطلع الكلمة نفسها نرجع من آخر خانة: لو جينا من القطر ناخذ الحرف.

مثال السلايد: X = ABCBDAB, Y = BDCABA

	∅	B	D	C	A	B	A
∅	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	**1**	1	1	1	2	2
C	0	1	1	**2**	2	2	2
B	0	1	1	2	2	**3**	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	**4**
B	0	1	2	2	3	4	4

```
Length = 4          LCS = BCBA     (BCAB and BDAB are also correct)
```

الخانات الذهبية: الحروف المأخوذة. الخانات الفاتحة: مسار الرجوع.

يتكرر في الميد

LCS of X = ACBAED and Y = ABCADF → length 4: ACAD or ABAD (472, 462, 451)

Matrix chain A = 4×5, B = 5×8, C = 8×3, D = 3×7 → 264, ((A(BC))D) (472, 451)

وقت التشغيل: LCS `O(mn)`، و Matrix chain `O(n³)`.